UET.CS2043 - Lập trình nâng cao

Thừa kế và Đa hình

Sử dụng lại trong OOP

- Chương trình có nhiều loại đối tượng có thông tin và hành vi tương tự, liên quan đến nhau:

  Person, Employee, Manager,...

- Nhu cầu tái sử dụng mã nguồn (code reuse)
  - Copy mã nguồn vào chương trình

    Thủ công, dễ nhầm, khó sửa lỗi,...
  - Sử dụng lại thông qua quan hệ "has – a" (composition)

    Chưa mềm dẻo, phải viết lại các giao diện,...
  - Sử dụng lại thông qua quan hệ "is – a" (inheritance)

    Tính đa hình - polymorphism

Quan hệ "has – a"

```
public class Person {
        private String name;
        private Date bithday;
        public String getName() { return name; }
}
```

```
public class Employee {
        private Person me;    //flexible?
        private double salary;
        public String getName() {
                return me.getName(); //flexible?
        }
}
```

```
public class Manager{
        private Employee me;  //flexible?
        private Employee assistant; //flexible?
        public void setAssistant(Employee e) {
                return assistant = e;
        }
}
```

```
…
Manager junior = new Manager();
Manager senior = new Manager();
//a manager can be an assistant of
//another manager
senior.setAssistant(junior);  //error
…
```

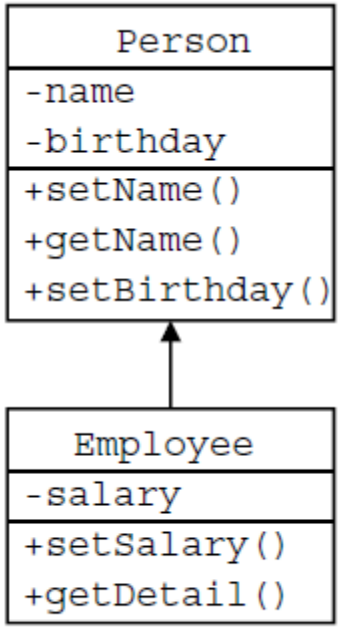
Kế thừa – Inheritance

- Sử dụng lại thông qua quan hệ “is – a”
- Thừa hưởng lại các thuộc tính và phương thức đã có
- Chi tiết hóa cho phù hợp với đối tượng mới
  - Thêm thuộc tính, phương thức mới
  - Hiệu chỉnh, cài đặt chi tiết phương thức được thừa kế
- Lớp cơ sở, lớp cha; lớp dẫn xuất, lớp con

Kế thừa trong Java

```
[public] class DeriveClass extends BaseClass{ }
```

```
public class Person {
                    String name;
                    Date birthday;
                    public void setName(String n) { name = n; }
                    public void setBirthday(Date d) { birthday = d; }
                    public String getName() { return name; }
}
…
public class Employee extends Person {
                    …
                    double salary;
                    public boolean setSalary (double sal) {
                              salary = sal;
                              return true;
                    }
                    …
}
```




```
Employee e = new Employee ();
e.setName("John");
e.setSalary(3.66);
System.out.print (e.getDetail());
```

Kiểm soát truy cập

- public, protected, “default” (packaged), private

	Same Class	Package	Subclass (same package)	Subclass (different package)	Universal
public	+	+	+	+	+
protected	+	+	+	+	
packaged	+	+			
private	+				

Mức truy cập protected

- Đối tượng thuộc lớp dẫn xuất truy cập được các *protected members (thuộc tính, phương thức) của lớp cơ sở*
- Với các đối tượng khác *che giấu thông tin*

```
public class Person {
                    protected String name;
                    protected int age;
                    ...
}
public class Employee extends Person {
                    double salary;
                    public String getDetail () {
                       String s = name + “, ”+ age;
                       s += “, “ + salary;
                       return s;
                    }
                    …
}
```

Private trong kế thừa

- Lớp dẫn xuất không kế thừa các private members của lớp cơ sở

thành phần bị ẩn

- Có thể truy cập các *thành phần private (bị ẩn)* thông qua các giao diện của lớp cơ sở

Đối tượng của lớp dẫn xuất có chứa thông tin thuộc tính *private*?

```
public class Person {
            private String name;
            private int age;
            public String getNam(){
                    return name;}
            public int getAge () {
                    return age;}
                    ...
}
public class Employee extends Person {
             double salary;
             public String getDetail () {
                //String s = name + “, ”+ age;
                String s = getNam() + “, “+ getAge();
                s += “, “ + salary;
                return s;
             }
                    …
}
```

Kế thừa: trong cùng gói, khác gói

Trong cùng gói

```
class Person {
        String name;
        int age;
        public String getNam(){
                return name;}
        public int getAge () {
                return age;}
        …
}
class Employee extends Person {
        double salary;
        …
        public String getDetail () {
                String s = name + “, ” + age;
                s += “, “ + salary;
                return s;
        }
        …
}
```

Khác gói

```
package comp;
public class Person {
        protected String name;
        protected int age;
        public String getNam(){
                return name;}
        public int getAge () {
                return age;}
        …
}
import comp.Person;
public class Employee extends Person {
        double salary;
        …
        public String getDetail () {
                String s = name + “, ” + age;
                s += “, “ + salary;
                return s;
        }
        …
}
```

Kế thừa từ gói khác

- Có thể kế thừa từ gói khác
  - Kế thừa thư viện của Java: ví dụ từ Applet
  - Kế thừa từ gói của các nhà phát triển khác
- Kế thừa mà không cần biết mã nguồn
  - Bảo mật mã nguồn
  - Nâng cao khả năng sử dụng lại

Kế thừa: Phương thức khởi tạo

- Lớp dẫn xuất không kế thừa các phương thức khởi tạo (*constr*)
- Lớp dẫn xuất:
  - Gọi constr mặc định của nó hoặc gọi *constr* chính lớp đó xây dựng

```
public class Point {
        protected int x, y;
        public Point (){}
        public Point (int xx, int yy){
                x = xx;
                y = yy;
        }
…
}
public class Circle extends Point {
        protected double radius;
        public Circle () {}
}
…
Point p1 = new Point (10, 10);
Circle c1 = new Circle ();
Circle c2 = new Circle (10, 10); //error
```

Lớp dẫn xuất: Phương thức khởi tạo

- Phương thức khởi tạo của lớp dẫn xuất:
  - Lớp cơ sở có phương thức khởi tạo (*constr*) mặc định: *có thể gọi hoặc không gọi* constr của lớp cơ sở
  - Gọi constr của lớp cơ sở thông qua từ khóa *super*

```
public class Point {
        protected int x, y;
        public Point (){}
        public Point (int xx, int yy){
                x = xx;
                y = yy;
        }
...
}
public class Circle extends Point {
        protected double radius;
        public Circle () {}
  public Circle (int xx, int yy, double r){
                super (xx, yy); //có thể bỏ câu lệnh này
                radius = r;
        }
}
```

Lớp dẫn xuất: Phương thức khởi tạo

- Phương thức khởi tạo của lớp dẫn xuất:
  - Lớp cơ sở không có *constr* mặc định: bắt buộc phải gọi *constr* của lớp cơ sở một cách tường minh
  - Câu lệnh super gọi *constr* của lớp cơ sở được đặt đầu tiên

```
public class Point {
        protected int x, y;
        public Point (int xx, int yy){
                x = xx;
                y = yy;
        }
...
}
public class Circle extends Point {
        protected double radius;
        public Circle () { super (0,0); }
  public Circle (int xx, int yy, double r){
                super (xx, yy); //bắt buộc phải gọi
                radius = r;
        }
}
```

Lớp dẫn xuất: Phương thức khởi tạo

- Thứ tự khởi tạo trong *constr* của lớp dẫn xuất:
  - Constr của lớp cơ sở được gọi
  - Các lệnh của constr lớp dẫn xuất được gọi tiếp theo

```
public class Point {
            protected int x, y;
            public Point (){
                System.out.println("Point constructor");
            }
…
}
public class Circle extends Point {
            protected double radius;
            public Circle (){
        System.out.println("Circle constructor");
            }
}
…
Circle c = new Circle ();

-----------------------------------
Point constructor
Circle constructor
```

Định nghĩa lại - overriding

- Có thể định nghĩa lại các phương thức được kế thừa (non-private) từ lớp cơ sở:
  - Chi tiết hóa, phù hợp với lớp dẫn xuất
- Khi gọi một phương thức của lớp dẫn xuất:
  - Thực hiện theo cơ chế liên kết động (dynamic binding)
  - Nếu lớp dẫn xuất định nghĩa lại phương thức, phiên bản định nghĩa lại phương thức được thực hiện
  - Nếu lớp dẫn xuất không định nghĩa lại phương thức đó, phương thức được định nghĩa từ lớp cơ sở "gần nhất" sẽ được gọi (bottom – up binding)

Định nghĩa lại - overriding

- Có thể gọi phương thức của lớp cha bằng từ khóa super
- Gọi phương thức của lớp cơ sở cao hơn (lớp ông, ...) như thế nào?
  - super.super...?
  - Thông qua gọi phương thức của lớp cha

```
class Person {
                    String name;
                    int age;
                    public String getDetail () {…}
                    ...
}
class Employee extends Person {
            double salary;
            …
            public String getDetail () {
               String s;
               s = super.getDetail() +“, “ + salary;
               return s;
            }
}
```

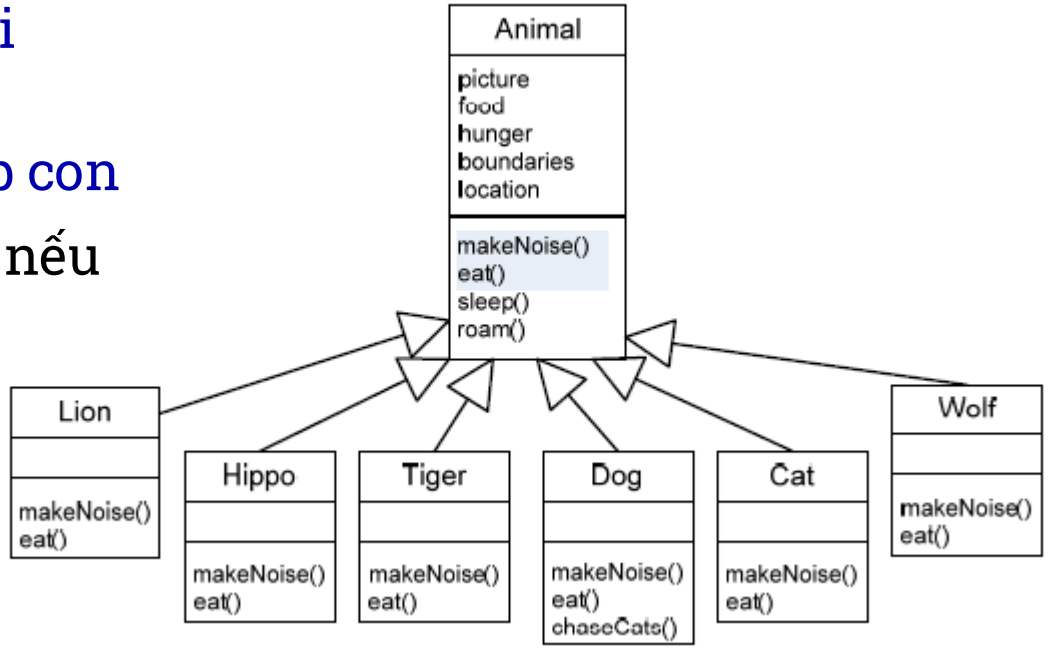
Định nghĩa lại - overriding

- Phạm vi truy cập
  - Phương thức được định nghĩa lại có phạm vi truy cập *không chặt hơn* phạm vi truy cập của lớp cơ sở
- Kiểu giá trị trả lại như nhau trong phương thức được định nghĩa lại
- Phương thức private
  - Không được kế thừa ở lớp dẫn xuất
  - Không được coi là *"định nghĩa lại"*

```
public class Parent{
        public void methodA() {}
        private int methodB() {
                return 0;
        }
...
}
public class Child extends Parent {
        protected void methodA() {} //error
        private void methodB() {}
}
```

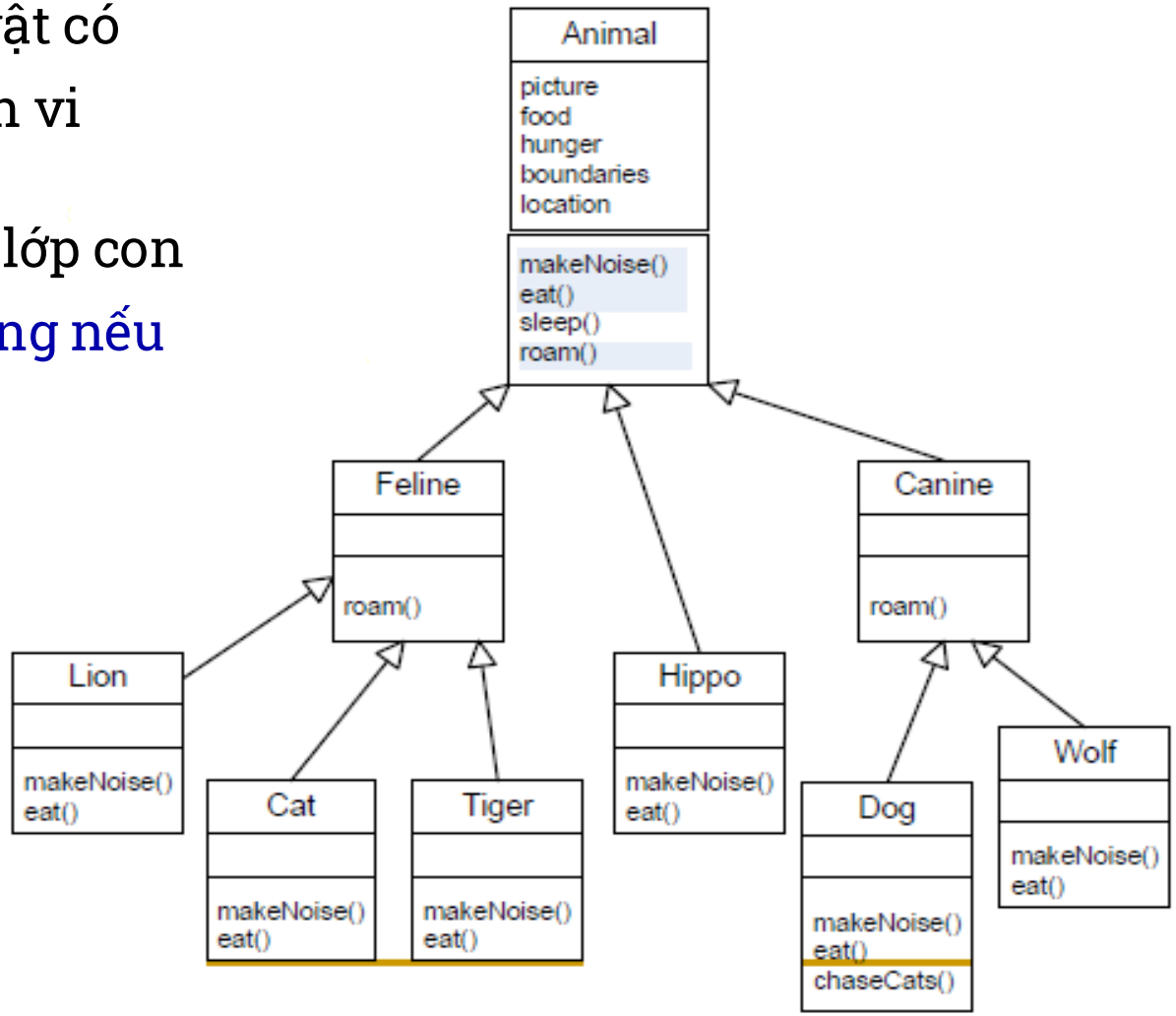
Thiết kế quan hệ kế thừa

- Tìm tất cả các đặc trưng chung mà các loại vật có
- Thiết kế lớp cha với tất cả thuộc tính và hành vi chung
- Thiết kế các thuộc tính và hành vi riêng của lớp con
- Nhóm tập nhỏ các lớp con vào một nhóm riêng nếu có thể.



Thiết kế quan hệ kế thừa

- Tìm tất cả các đặc trưng chung mà các loại vật có
- Thiết kế lớp cha với tất cả thuộc tính và hành vi chung
- Thiết kế các thuộc tính và hành vi riêng của lớp con
- Nhóm tập nhỏ các lớp con vào một nhóm riêng nếu có thể.



Từ khóa final

- Thuộc tính final
  - Hằng số, gán giá trị một lần không thay đổi được.

    không cho thay đổi giá trị của biến
- Phương thức final
  - Không cho phép định nghĩa lại ở lớp dẫn xuất

    không cho ghi đè (overriding) phương thức
- Tham số final
  - Tham số truyền vào không gán thay đổi giá trị

    nhưng cho phép thay đổi giá trị thuộc tính của tham chiếu đối tượng
- Lớp final
  - Không định nghĩa được lớp dẫn xuất

    lớp final không cho phép có lớp con

Tham số final

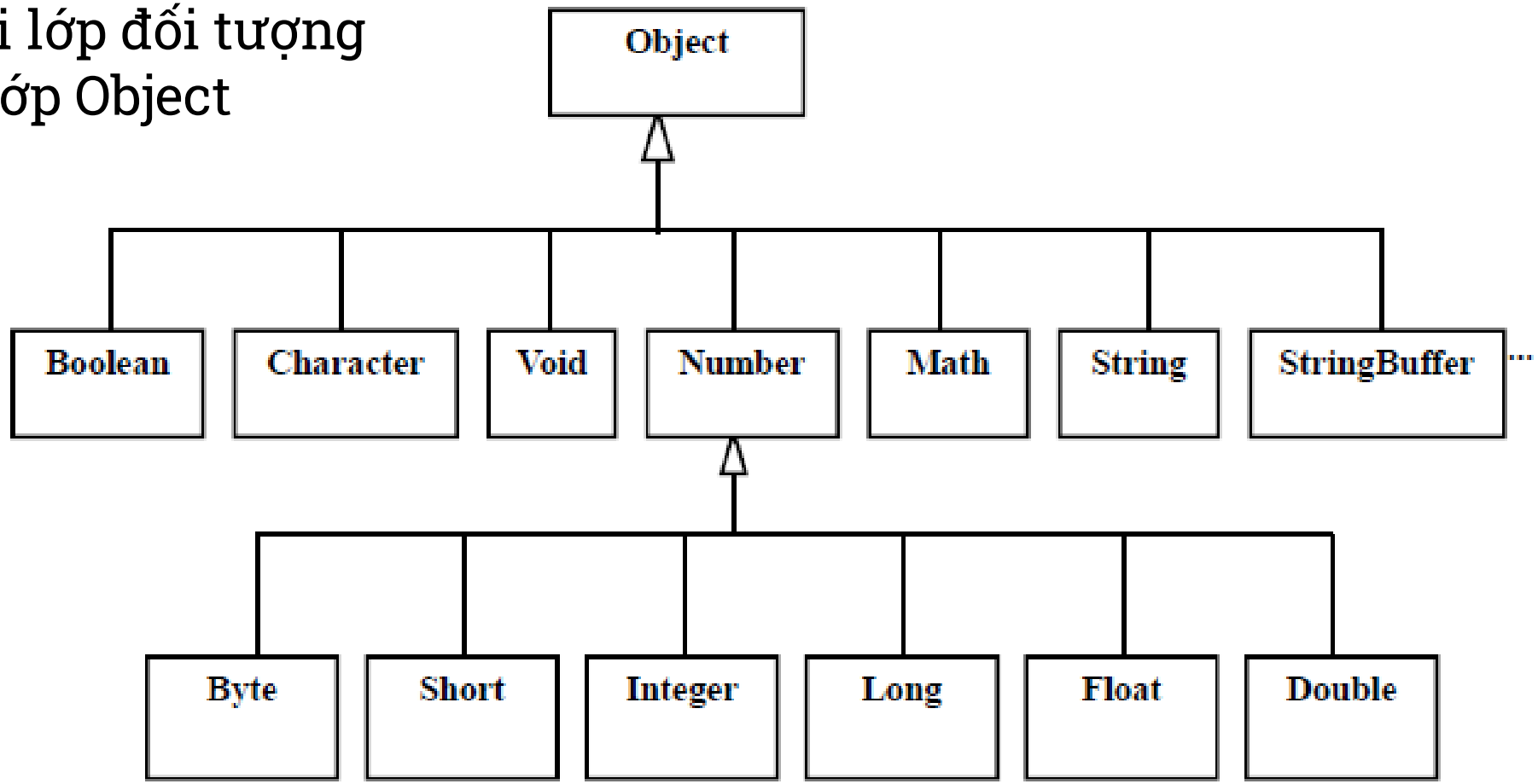
```
class MyDate {
                    int year, month, day;
                    public MyDate(int y, int m, int d) {
                        year = y;
                        month = m;
                        day = d;
                    }
                    public void copyTo (final MyDate d) {
                        d.year = year;
                        d.month = month;
                        d.day = day;
                        //d = new MyDate(year, month, day);
                    }
…
}
```

Tham số truyền vào không gán thay đổi giá trị

nhưng cho phép thay đổi giá trị thuộc tính của tham chiếu đối tượng

Lớp Object

Trong Java, mọi lớp đối tượng đều thừa kế từ lớp Object



Thảo luận: “is-a” và “has-a”

```java
// This class is BRITTLE due to implementation details of the parent
class CountingHashSet<E> extends HashSet<E> {
    private int addCount = 0;

    @Override
    public boolean add(E e) {
        addCount++;
        return super.add(e);
    }

    @Override
    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c); // HashSet's addAll calls add() internally!
    }

    public int getAddCount() { return addCount; }
}

// TEST: Adding 3 elements results in addCount = 6 (Double Counting)
// Because HashSet.addAll internally calls add() for each element.
```

```java
// Durable: Relies on the Interface (Set), not the Implementation (HashSet)
class DurableCountingSet<E> {
    private final Set<E> set; // Composition: "Has-a" Set
    private int addCount = 0;

    public DurableCountingSet(Set<E> set) {
        this.set = set;
    }

    public boolean add(E e) {
        addCount++;
        return set.add(e);
    }

    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return set.addAll(c); // Independent of set's internal implementation
    }

    public int getAddCount() { return addCount; }
}
```

Khái niệm đa hình - polymorphism

- Kiểu dữ liệu tham chiếu
  - Tham chiếu trỏ đến nhiều loại đối tượng khác nhau
  - Tham chiếu có thể *trỏ đến đối tượng là dẫn xuất* của kiểu dữ liệu được khai báo cho tham chiếu đó
- Thông điệp được gọi
  - Liên kết động – dynamic binding: thông điệp được gọi xác định "động", tùy thuộc vào loại đối tượng được tạo ra.
  - Cơ chế giải nghĩa thông điệp: liên kết động, không phải luôn luôn thực hiện phương thức theo kiểu dữ liệu đã khai báo

Đa hình – kiểu đối tượng tham chiếu trỏ tới

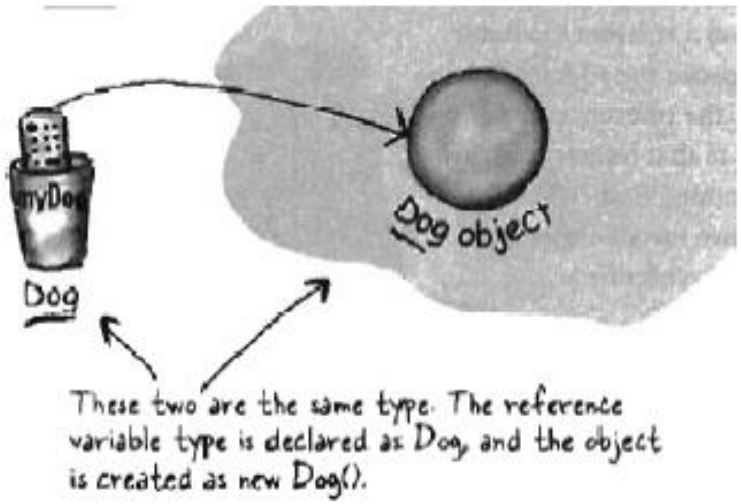
- Thông thường

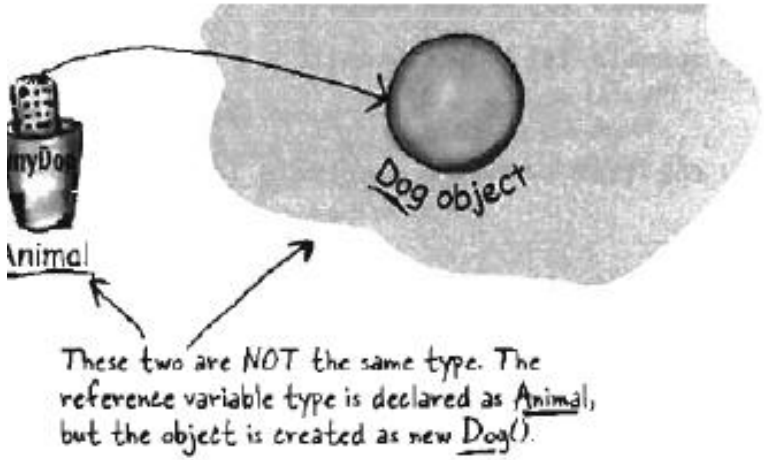
  Dog d = new Dog();

- Đa hình cho phép

  Animal d = new Dog();

  Tham chiếu có thể trỏ đến đối tượng là dẫn xuất





Đa hình – mảng các đối tượng khác nhau

- Một mảng khai báo kiểu Animal nhưng có thể *chứa các đối tượng là dẫn xuất* của Animal

```
Animal [] animals = new Animal[5];

Animals[0] = new Dog();
Animals[1] = new Cat();
Animals[2] = new Lion();
Animals[3] = new Hippo();
Animals[4] = new Wolf();

for (int i=0; i< animals.length; i++)
             animals[i].makeNoise();

//Dynamic binding: makeNoise() of each object Dog, Cat,
//Lion, Hippo, Wolf is executed.
```

Không cần biết trong mảng có những đối tượng loại nào…

Đa hình – tham số của hàm

- Tham số kiểu Animal nhưng có thể *truyền vào tham chiếu đối tượng là dẫn xuất* của Animal

```
class Vet {
    …
    public void giveShot (Animal a){
                    a.makeNoise();
                }
}
…

Vet v = new Vet ();
Dog d = new Dog();
Cat c = new Cat();
v.giveShot (d);    //Dog makeNoise()
v.giveShot (c);    //Cat makeNoise()
```

Đa hình – giải nghĩa thông điệp động

```
class Animal {
            String name;
    …
    public void makeNoise (){
                System.out.println("Animal noise…");
            }
    public void introduce (){
                makeNoise();
                System.out.println("I am "+ name);
            }
}
class Cat extends Animal{
    public void makeNoise (){
                System.out.println("Meo meo…");
            }
}
class Dog extends Animal{
    public void makeNoise (){
                System.out.println("Gau gau…");
            }
}
```

```
Animal pet1, pet2;
pet1 = new Cat("Tom cat");
pet2 = new Dog("Lulu dog");
pet1.introduce();
pet2.introduce();
```

//Meo meo… I am Tom cat
//Gau gau… I am Lulu dog

private void makeNoise()?
Liên kết tĩnh – static binding

//Animal noise… I am Tom cat
//Animal noise… I am Lulu dog

Đa hình – ưu điểm

- Viết mã chương trình một lần
  - Áp dụng cho nhiều kiểu đối tượng

Liên kết động – dynamic binding

  - Không phải thay đổi mã nguồn khi tạo ra các lớp dẫn xuất mới

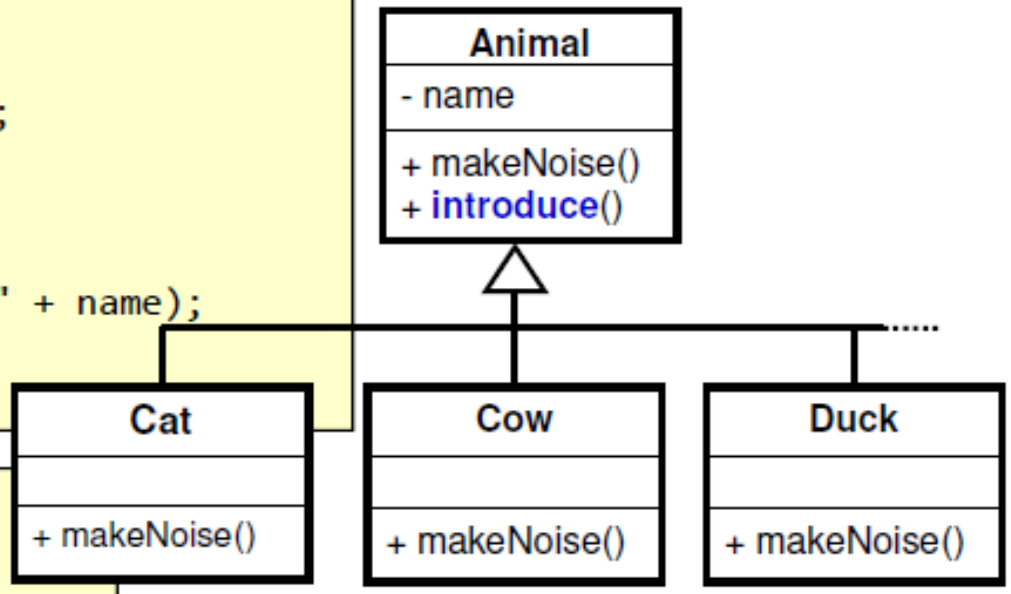
Nâng cấp phiên bản, kiến trúc phần mềm linh hoạt,...

```
class Vet {
    …
    public void giveShot (Animal a){
                    a.makeNoise();
                }
}
Vet v = new Vet ();
Dog d = new Dog();
Cat c = new Cat();
v.giveShot (d);    //Dog makeNoise()
v.giveShot (c);    //Cat makeNoise()
```

```
Animal [] animals = new Animal[5];
…
for (int i=0; i< animals.length; i++)
                animals[i].makeNoise();
```

Đa hình – ưu điểm

```
class Animal {
   ...
   public void makeNoise() {
      System.out.print ("Hmm.");
   }
   public void introduce() {
      makeNoise();
      System.out.println(" I'm " + name);
   }
}
```




```
class Pig extends Animal {
   public void makeNoise() {
      System.out.print("Oi oi...");
   }
}
```

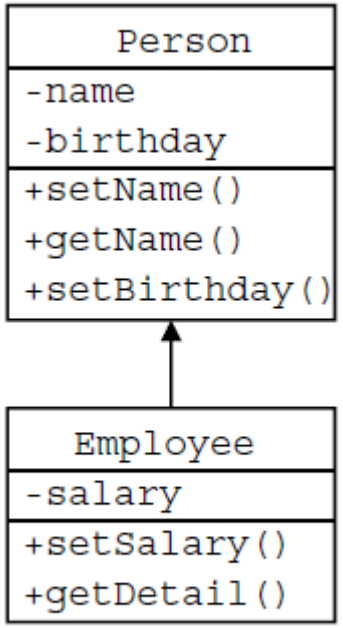
```
class Duck extends Animal {
   public void makeNoise() {
      System.out.print("Quack quack...");
   }
```

i.e., Bổ sung thêm nhiều lớp dẫn xuất của Animal mà hàm introduce() luôn hoạt động đúng

Mã nguồn chạy đúng với các phiên bản nâng cấp trong tương lai.

Cho phép xây dựng các hàm có tính chất khuôn mẫu.

Chuyển kiểu lên - upCasting

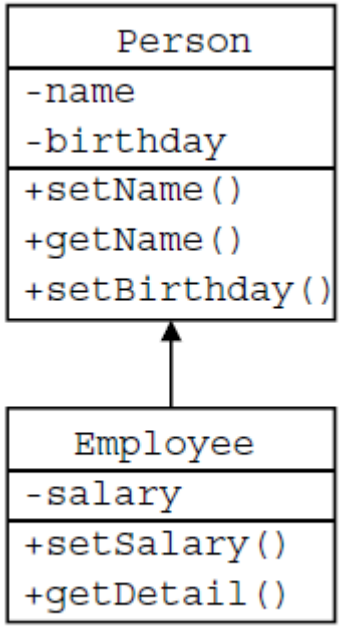



```
Person p;
Employee e = new Employee();
p = (Person) e;          //explicit up casting
p.setName (…);
p.setSalary(…);          //compile error
```

```
Person p;
Employee e = new Employee();
p = e;                   //implicit up casting
p.setName (…);
p.setSalary(…);          //compile error
```

Sự khác nhau giữa explicit up casting và implicit up casting?

Chuyển kiểu xuống - downCasting




```
Person p;
Employee e = new Employee();
p = (Person) e;                    //explicit up casting
p.setName (…);
p.setSalary(…);  //compile error
Employee ee = (Employee) p;        //down casting
Manager m = (Manager) ee;          //compile error
```

```
Person p = new Manager();
Employee e = (Employee) p;         //up or down casting?
```

Chỉ down casting trên những tham chiếu đối tượng đã up casting

Liên kết tĩnh, liên kết động

- Liên kết tĩnh – static binding: lời gọi phương thức được quyết định khi biên dịch
  - Ưu điểm về tốc độ
  - final, private method
- Liên kết động – dynamic binding: lời gọi phương thức được quyết định khi thực hiện (executed), phiên bản của phương thức phù hợp với đối tượng được gọi
  - Java mặc định áp dụng liên kết động (ngoại trừ final/private method)

không phụ thuộc vào kiểu dữ liệu khi khai báo,
phương thức được định nghĩa ở đâu,...

Đa hình – gọi phương thức trong constructor

```
class Shape {
            public Shape() {
                        draw();
    }
    public void draw() {}
}
class Point extends Shape {
    protected int x, y;

    public Point (int xx, int yy) {
                        x = xx; y = yy;
    }
    public void draw() {
                        System.out.println("(" + x + "," + y + ")");
    }
}
...
Point p = new Point(10, 10);
-----------------------
(10,10)
```

```
private void draw() {}
```

Đa hình – private method

```
class Base {
    private void f() {
                System.out.println("base f()");
    }
            public void show() { f(); }
}
public class Derived extends Base {
            private void f() {
                System.out.println("derived f()");
    }
    public static void main (String args[]) {
                Derived d = new Derived();
                Base b = d;
                b.show();
    }
}
```

Liên kết tĩnh → "base f()"

Toán tử instanceof

```
public class Employee extends Person {}
public class Student extends Person {}
        ...
        public doSomething (Person e) {
                if (e instanceof Employee) {
                ...
                } else if (e instanceof Student) {
                ...
                } else {
                ...
                }
        }
}
```

Lớp trừu tượng – abstract class

- Có thể tạo ra các lớp cơ sở (khuôn mẫu) để các lớp dẫn xuất kế thừa mà không tạo ra các đối tượng thực của lớp
  - i.e., lớp Dog, Cat, Cow,... kế thừa từ lớp Animal; Rectangle, Circle,... kế thừa từ lớp Shape

    Thực tế không có đối tượng nào là Animal, Shape,...
- Khai báo lớp trừu tượng – abstract class
  - Không thể tạo ra đối tượng
  - Xây dựng lớp khuôn mẫu với các thuộc tính và hành vi mà các lớp dẫn xuất bắt buộc phải có

```
abstract class Shape {
        protected int x, y;
        Shape(int _x, int _y) {
                x = _x;
                y = _y;
        }
}
class Circle extends Shape {...}
Shape s = new Shape(10, 10)
// compile error
Shape s1 = new Circle();
```

Phương thức trừu tượng

- Để thống nhất giao diện, có thể khai báo các phương thức trừu tượng (abstract method) tại các lớp cơ sở và cài đặt chi tiết tại các lớp dẫn xuất.
  - Các lớp dẫn xuất cài đặt các phiên bản khác nhau của phương thức trừu tượng được kế thừa.
- Phương thức trừu tượng

→ Bắt buộc phải định nghĩa lại tại lớp dẫn xuất.

```
abstract class Shape {
   protected int x, y;
   abstract public void erase();
   abstract public void draw();
   public void moveTo (int x1, int y1)
{...}
}
```

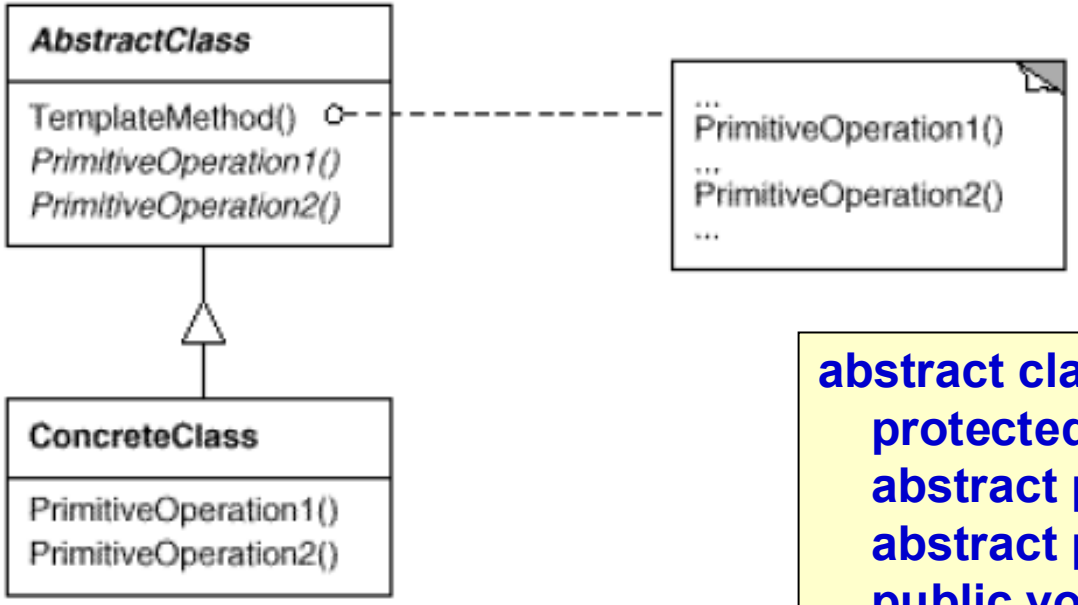
Phương thức trừu tượng

```
abstract class Shape {
    protected int x, y;
    abstract public void erase();
    abstract public void draw();
    public void moveTo (int x1, int y1) {…}
}
```

```
class Circle extends Shape {
                int r;
    public Circle(int _x, int _y, int _r) {
                        super (_x, _y);
                        r = _r;
                        draw();
    }
   public void erase() {
                        System.out.println("Erase at (" + x +
            "," + y + ")");
    }
    public void draw() {
                        System.out.println("Draw at (" + x +
            "," + y + ")");
    }
}
```

→ Phương thức trừu tượng khai báo trong lớp .... trừu tượng.
→ Lớp trừu tượng có phương thức thường và phương thức trừu tượng.
→ Phải định nghĩa phương thức trừu tượng trong lớp dẫn xuất.

Phương thức khuôn mẫu - template method



Template method

```
abstract class Shape {
    protected int x, y;
    abstract public void erase();
    abstract public void draw();
    public void moveTo (int x1, int y1) {
                    erase();
                    x = x1;
                    y = y1;
                    draw();
    }
}
```

Giao diện - interface

- interface là mức trừu tượng cao
- interface bao gồm:
  - Phương thức trừu tượng – abstract method
  - Hằng số: mặc định là public static final
  - Mặc định là public
- Từ khóa: interface và implements

```
interface Config {
                int MAX_CONN = 20;
                void display();  //it will be public
}
…
class Control implements Config {…}
```

Lớp cài đặt giao diện

Lớp thường cài đặt giao diện

```
interface Action {
        void moveTo(int x, int y);
        void erase();
        void draw();
}
class Circle1 implements Action {
        int x, y, r;
        Circle1(int _x, int _y, int _r) { ... }
        public void erase() {...}
        public void draw() {...}
        public void moveTo(int x1, int y1)
        { ... }
}
```

Lớp trừu tượng cài đặt giao diện

```
abstract class Shape implements Action {
        protected int x, y;
        public Shape() {...}
        public Shape(int _x, int _y) {...}
        public void moveTo(int x1, int y1) {
                erase();
                x = x1;
                y = y1;
                draw();
        }
        //public void erase() {...}
        //public void draw() {...}
}
```

Các giao diện kế thừa

```
interface I1 {...}
interface I2 {...}
interface I3 extends I1, I2 {...}
//Giao diện kế thừa giao diện

abstract class A1 {...}
abstract class A2 extends A1 implements
I1, I2 {...}
```

```
interface I1 { void f(); }
interface I2 { int f(int i); }
interface I3 { int f(); }
class C {
        public int f() { return 1; }
}
class C2 implements I1, I2 {
        public void f() {}
        public int f(int i) { return 1; } //overloaded
}
class C3 extends C implements I2 {
        public int f(int i) { return 1; } // overloaded
}
class C4 extends C implements I3 {
        public int f() { return 1; } // Identical, no problem
}
class C5 extends C implements I1 {}
interface I4 extends I1, I3 {}
```

Xung đột khi cài đặt giao diện

Giao diện khai báo như 1 kiểu dữ liệu

```
interface CanFight {
        void fight();
}
interface CanSwim {
        void swim();
}
interface CanFly {
        void fly();
}
class ActionCharacter {
        public void fight() {}
}
```

```
class Hero extends ActionCharacter implements CanFight,
CanSwim, CanFly {
        public void swim() {}
        public void fly() {}
}
public class Adventure {
        public static void t(CanFight x) { x.fight(); }
        public static void u(CanSwim x){ x.swim();}
        public static void v(CanFly x) { x.fly(); }
        public static void w(ActionCharacter x) { x.fight(); }
        public static void main(String[] args) {
        Hero h = new Hero();
                t(h); // Treat it as a CanFight
                u(h); // Treat it as a CanSwim
                v(h); // Treat it as a CanFly
                w(h); //Treat it as an ActionCharacter
        }
}
```

Truyền vào hàm các tham chiếu đối tượng có cài đặt giao diện khai báo